Royaume du Maroc

Ministère de l'Éducation Nationale, du Préscolaire et des Sports

وزارة التربية الوطنية
والتعليم الأولي والرياضة

ALKendi
CENTRE DE PRÉPARATION BTS

Rapport de Séance

Matière : Développement Web

Les Fonctions en PHP

Réalisé par :
Rayan El Habib

Encadré par :
Dr. Ayman NAIM

Séance tenue le :
20 November 2025

Année académique : 2025/2026

Table des matières

Introduction **2**

1 Introduction aux fonctions **3**
1.1 Exercice 1 : Définition d'une fonction avec function 3
1.2 Exercice 2 : Appel d'une fonction 6

2 Création et appel de fonctions **8**
2.1 Exercice 3 : Passage de paramètres dans une fonction 8
2.2 Exercice 4 : Valeurs par défaut dans les paramètres 10
2.3 Exercice 5 : Fonctions avec un nombre variable d'arguments 12

3 Passage de paramètres et retour de valeurs **14**
3.1 Exercice 6 : Passage de variables par valeur et par référence 14
3.2 Exercice 7 : Retour d'une valeur avec return 16
3.3 Exercice 8 : Fonctions anonymes et callbacks 17

4 Fonctions prédéfinies **20**
4.1 Exercice 9 : Fonctions mathématiques 20
4.2 Exercice 10 : Fonctions de chaîne 22
4.3 Exercice 11 : Fonctions de tableau 24
4.4 Exercice 12 : Fonctions sur les fichiers 26

Introduction

Objectif de la Séance

L'objectif de cette séance est de **comprendre** et d'**appliquer** les fonctions en PHP, y compris la création et l'appel de fonctions, le passage de paramètres, et les fonctions prédéfinies. À travers des exercices pratiques, vous apprendrez à maîtriser ces mécanismes essentiels pour le développement en PHP.

Une base solide en fonctions est la clé d'un code PHP professionnel

Chapitre 1

Introduction aux fonctions

1.1 Exercice 1 : Définition d'une fonction avec function

1.1

Comment définit-on une fonction en PHP avec le mot-clé function ?

Réponse : En PHP, on définit une fonction en utilisant le mot-clé `function` suivi du nom de la fonction et de parenthèses pour les éventuels paramètres.

Exemple :

```php
function nomDeLaFonction() {
    // Corps de la fonction
    echo "Bonjour !";
}
```

1.2

Quelle est la syntaxe correcte pour nommer une fonction en PHP ?

Réponse : Le nom d'une fonction doit respecter les règles suivantes :

- Commencer par une lettre ou un underscore
- Ne contenir que des lettres, chiffres et underscores
- Être sensible à la casse
- Ne pas être un mot-clé réservé du PHP

Exemples :

```php
// Syntaxes valides
function maFonction() {}
function _traitementDonnees() {}
function calculerMoyenne() {}

// Syntaxes invalides
function 123maFonction() {}  // Commence par un chiffre
function ma-fonction() {}    // Contient un tiret
```

1.3

Quelle est la différence entre une fonction déclarée et une fonction anonyme ?

Réponse :

- **Fonction déclarée :** Définie avec un nom et disponible dans toute la portée
- **Fonction anonyme :** Sans nom, stockée dans une variable ou passée comme paramètre

Exemples :

```php
// Fonction d clar e
function saluer() {
    echo "Bonjour";
}

// Fonction anonyme
$saluer = function() {
    echo "Bonjour";
};
```

1.4

Que se passe-t-il si deux fonctions sont définies avec le même nom dans un même script PHP ?

Réponse : PHP génère une erreur fatale :

```
Fatal error: Cannot redeclare functionName()
```

Les noms de fonctions doivent être uniques dans la même portée.

1.5

Comment documenter efficacement une fonction avec des commentaires pour qu'elle soit lisible et compréhensible ?

Réponse : Utiliser la syntaxe PHPDoc pour une documentation professionnelle :

Exemple :

```php
/**
 * Calcule la moyenne d'un tableau de nombres
 *
 * @param array $nombres Tableau de nombres
 * @return float La moyenne calcul e
 * @throws InvalidArgumentException Si le tableau est vide
 */
function calculerMoyenne(array $nombres): float {
    if (empty($nombres)) {
        throw new InvalidArgumentException("Le tableau ne peut pas  tre  vide");
    }
    return array_sum($nombres) / count($nombres);
}
```

1.2 Exercice 2 : Appel d'une fonction

2.1

Comment appelle-t-on une fonction définie en PHP ?

Réponse : En utilisant son nom suivi de parenthèses :

Exemples :

```php
maFonction();  // Sans param tres
maFonction($param1, $param2);  // Avec param tres
```

2.2

Pourquoi est-il important de nommer les fonctions de manière significative ?

Réponse : Pour les raisons suivantes :

- Améliorer la lisibilité du code
- Faciliter la maintenance
- Réduire les erreurs
- Permettre une compréhension rapide de la fonction

2.3

Quels sont les risques d'un mauvais nommage de fonction dans un programme de grande taille ?

Réponse : Les risques incluent :

- Difficulté de maintenance
- Augmentation des bugs
- Temps de développement accrued
- Complexité pour les nouveaux développeurs

2.4

Que se passe-t-il si on essaie d'appeler une fonction avant de l'avoir définie dans le code ?

Réponse : PHP génère une erreur :

```
Fatal error: Uncaught Error: Call to undefined function functionName()
```

La fonction doit être définie avant son appel.

Chapitre 2

Création et appel de fonctions

2.1 Exercice 3 : Passage de paramètres dans une fonction

3.1

Comment passer des paramètres dans une fonction en PHP ?

Réponse : En les déclarant entre les parenthèses :

Exemple :

```
function afficherNom($nom, $prenom) {
    echo "Nom : $nom, Pr nom : $prenom";
}
```

3.2

Quelle est la différence entre passer un paramètre par valeur et par référence ?

Réponse :

— **Par valeur :** Une copie de la variable est créée

— **Par référence :** La fonction travaille sur la variable originale

Exemple :

```
// Par valeur (d faut)
function incrementerValeur($x) {
    $x++;
}

// Par r f rence
function incrementerReference(&$x) {
    $x++;
}

$nombre = 5;
incrementerValeur($nombre);  // $nombre reste 5
incrementerReference($nombre); // $nombre devient 6
```

3.3

Expliquez l'importance de la gestion des paramètres dans des applications évolutives.

Réponse : Une bonne gestion permet :

— La flexibilité pour les évolutions futures

— La rétrocompatibilité

— Une meilleure maintenabilité

— La réduction des erreurs

3.4

Comment utiliser les types de paramètres pour éviter les erreurs de type ?

Réponse : En utilisant le typage des paramètres :

Exemple :

```
function additionner(int $a, int $b): int {
    return $a + $b;
}

// PHP g n rera une TypeError si les types ne
    correspondent pas
```

2.2 Exercice 4 : Valeurs par défaut dans les paramètres

4.1

Comment définir une valeur par défaut pour un paramètre de fonction ?

Réponse : En utilisant le signe = après le paramètre :

Exemple :

```
function saluer($nom = "Invit ") {
    echo "Bonjour $nom !";
}
```

4.2

Dans quels cas est-il avantageux d'utiliser des valeurs par défaut ?

Réponse : Pour :

- Paramètres optionnels
- Rétrocompatibilité
- Simplification des appels courants

4.3

Que se passe-t-il si une fonction utilise des valeurs par défaut et que tous les paramètres ne sont pas spécifiés ?

Réponse : Les paramètres non fournis prennent leur valeur par défaut. Si aucun défaut n'est défini, PHP génère une erreur.

4.4

Est-il possible de mélanger paramètres obligatoires et facultatifs dans une fonction ? Expliquez avec un exemple.

Réponse : Oui, mais les paramètres obligatoires doivent venir en premier :

Exemple :

```
// Correct
function configurer($obligatoire, $optionnel = "d faut") {
    // ...
}

// Incorrect - g n re une erreur
function configurer($optionnel = "d faut", $obligatoire) {
    // ...
}
```

2.3 Exercice 5 : Fonctions avec un nombre variable d'arguments

5.1

Comment utiliser func_get_args() pour gérer des arguments multiples ?

Réponse :

Exemple :

```
function somme() {
    $args = func_get_args();
    return array_sum($args);
}

echo somme(1, 2, 3, 4); // Affiche 10
```

5.2

Quels sont les avantages de cette fonctionnalité ?

Réponse :

- Flexibilité du nombre d'arguments
- Compatibilité avec différents cas d'usage
- Évite la surcharge de fonctions

5.3

Donnez un exemple pratique où des arguments variables sont particulièrement utiles.

Réponse : Dans une fonction de logging :

Exemple :

```
function logger($niveau, ...$messages) {
    $timestamp = date('Y-m-d H:i:s');
    foreach ($messages as $message) {
        echo "[$timestamp] [$niveau] $message\n";
    }
}

logger('INFO', 'D marrage', 'Configuration charg e');
logger('ERROR', 'Erreur base de donn es');
```

5.4

Comment utilise-t-on l'opérateur ... pour simplifier la gestion d'arguments variables ?

Réponse : Avec l'opérateur de déstructuration :

Exemple :

```
function concatener(...$chaines) {
    return implode(' ', $chaines);
}

echo concatener('Bonjour', 'le', 'monde'); // "Bonjour le
    monde"
```

Chapitre 3

Passage de paramètres et retour de valeurs

3.1 Exercice 6 : Passage de variables par valeur et par référence

6.1

Quelle est la différence entre le passage par valeur et par référence en PHP ?

Réponse :

- **Par valeur :** Crée une copie, modifications locales
- **Par référence :** Travaille sur l'original, modifications permanentes

6.2

Comment modifier une variable en dehors de la fonction en utilisant le passage par référence ?

Réponse : En utilisant & devant le paramètre :

Exemple :

```
function modifierVariable(&$variable) {
    $variable .= " modifi e";
}

$texte = "Valeur originale";
modifierVariable($texte);
echo $texte; // "Valeur originale modifi e"
```

6.3

Quelles sont les implications en mémoire du passage par référence ?

Réponse :

— Économie mémoire pour les grandes structures
— Risque d'effets de bord non désirés
— Meilleures performances pour les gros objets

6.4

Quand est-il préférable d'utiliser le passage par référence plutôt que le passage par valeur ?

Réponse : Pour :

— Grands tableaux ou objets
— Fonctions de modification directe
— Optimisation des performances
— Retour multiple de valeurs

3.2 Exercice 7 : Retour d'une valeur avec return

7.1

Comment utiliser l'instruction return pour renvoyer une valeur d'une fonction ?

Réponse :

Exemple :

```
function calculerCarre($nombre) {
    return $nombre * $nombre;
}

$resultat = calculerCarre(5); // 25
```

7.2

Dans quels cas est-il utile de renvoyer des valeurs depuis une fonction ?

Réponse : Pour :

- Calculs et transformations
- Validation de données
- Récupération d'informations
- Chaînage de fonctions

7.3

Que se passe-t-il si une fonction ne retourne pas de valeur ?

Réponse : La fonction retourne `null` par défaut.

7.4

Est-il possible de retourner plusieurs valeurs depuis une fonction ? Expliquez comment.

Réponse : Oui, en utilisant un tableau ou un objet :

Exemple :

```
// Avec un tableau
function obtenirCoordonnees() {
    return ['x' => 10, 'y' => 20];
}

// Avec list()
list($x, $y) = obtenirCoordonnees();

// Avec d structuration (PHP 7.1+)
[$x, $y] = obtenirCoordonnees();
```

3.3 Exercice 8 : Fonctions anonymes et callbacks

8.1

Qu'est-ce qu'une fonction anonyme en PHP et comment l'utiliser ?

Réponse : Une fonction sans nom, assignée à une variable :

Exemple :

```
$addition = function($a, $b) {
    return $a + $b;
};
echo $addition(5, 3); // 8
```

8.2

Donnez un exemple d'utilisation de callbacks en PHP.

Réponse :

Exemple :

```php
// Avec array_map
$nombres = [1, 2, 3, 4];
$carres = array_map(function($n) {
    return $n * $n;
}, $nombres);

// Avec usort
usort($tableau, function($a, $b) {
    return $a - $b;
});
```

8.3

Quelles sont les différences entre les fonctions anonymes et les fonctions de rappel (callbacks) en termes de syntaxe et d'utilisation ?

Réponse :

- **Fonction anonyme :** Syntaxe avec `function()`
- **Callback :** Peut être une chaîne (nom fonction), un tableau, ou une closure
- Les fonctions anonymes sont un type de callback

8.4

Comment passer une fonction anonyme comme argument à une autre fonction en PHP ?

Réponse : Directement comme paramètre : **Exemple :**

```
function traiterDonnees($donnees, $callback) {
    foreach ($donnees as $donnee) {
        $resultat = $callback($donnee);
        echo $resultat . "\n";
    }
}

traiterDonnees([1, 2, 3], function($n) {
    return $n * 2;
});
```

Chapitre 4

Fonctions prédéfinies

4.1 Exercice 9 : Fonctions mathématiques

9.1

Comment utiliser la fonction abs() pour obtenir la valeur absolue ?

Réponse : Exemple :

```
echo abs(-5);     // 5
echo abs(3.14);   // 3.14
echo abs(0);      // 0
```

9.2

Quelle est l'utilité des fonctions round(), ceil() et floor() en PHP ?

Réponse :

- `round()` : Arrondi au plus proche
- `ceil()` : Arrondi vers le haut
- `floor()` : Arrondi vers le bas

Exemple :

```
echo round(3.6);   // 4
echo ceil(3.2);    // 4
echo floor(3.8);   // 3
```

9.3

Quand utiliseriez-vous floor() au lieu de round() dans une application ?

Réponse : Pour :

- Calcul de prix plancher
- Division entière
- Pagination (nombre de pages)
- Arrondi systématique vers le bas

9.4

Donnez un exemple d'utilisation combinée de fonctions mathématiques pour formater une valeur.

Réponse :

Exemple :

```
function formaterPrix($prix) {
    $prixArrondi = round($prix, 2);
    $prixFormate = number_format($prixArrondi, 2, ',', ' ');
    return $prixFormate . '    ';
}

echo formaterPrix(1234.5678); // "1 234,57    "
```

4.2 Exercice 10 : Fonctions de chaîne

10.1

Comment obtenir la longueur d'une chaîne avec strlen() ?

Réponse :

Exemple :

```
$texte = "Bonjour";
echo strlen($texte); // 7
```

10.2

Expliquez l'utilisation de str_replace() et substr().

Réponse :

Exemple :

```php
// str_replace() - remplacement
$texte = "Hello World";
$nouveau = str_replace("World", "PHP", $texte);
echo $nouveau; // "Hello PHP"

// substr() - extraction
$texte = "Bonjour le monde";
echo substr($texte, 0, 7);    // "Bonjour"
echo substr($texte, 8, 2);    // "le"
echo substr($texte, -5);      // "monde"
```

10.3

Comment combiner str_replace() et substr() pour manipuler des chaînes plus complexes ?

Réponse :

Exemple :

```php
function formaterTelephone($telephone) {
    // Extraire les parties
    $indicatif = substr($telephone, 0, 3);
    $partie1 = substr($telephone, 3, 2);
    $partie2 = substr($telephone, 5, 2);
    $partie3 = substr($telephone, 7, 2);
    $partie4 = substr($telephone, 9, 2);

    // Formater
    return "+$indicatif $partie1 $partie2 $partie3 $partie4"
        ;
}

echo formaterTelephone("212612345678"); // "+212 61 23 45
    67"
```

10.4

Expliquez l'utilisation de `strpos()` pour rechercher une sous-chaîne dans une chaîne.

Réponse :

Exemple :

```
$texte = "Bienvenue au cours de PHP";
$position = strpos($texte, "PHP");

if ($position !== false) {
    echo "Trouv     la position : $position"; // 21
} else {
    echo "Non trouv ";
}
```

4.3 Exercice 11 : Fonctions de tableau

11.1

Comment fusionner deux tableaux avec array_merge() ?

Réponse :

Exemple :

```
$tableau1 = ['a', 'b', 'c'];
$tableau2 = ['d', 'e', 'f'];
$resultat = array_merge($tableau1, $tableau2);
// ['a', 'b', 'c', 'd', 'e', 'f']
```

11.2

Que font les fonctions array_slice() et array_keys() ?

Réponse :

Exemple :

```
// array_slice() - extraction
$tableau = ['a', 'b', 'c', 'd', 'e'];
$extrait = array_slice($tableau, 1, 3);
// ['b', 'c', 'd']

// array_keys() - cl s
$associatif = ['nom' => 'Ali', 'age' => 25];
$cles = array_keys($associatif);
// ['nom', 'age']
```

11.3

Expliquez un exemple d'utilisation pratique de array_slice() pour paginer des résultats.

Réponse :

Exemple :

```
function paginer($donnees, $page, $parPage) {
    $debut = ($page - 1) * $parPage;
    return array_slice($donnees, $debut, $parPage);
}

$donnees = range(1, 100); // [1, 2, 3, ..., 100]
$page1 = paginer($donnees, 1, 10); // [1     10]
$page2 = paginer($donnees, 2, 10); // [11     20]
```

11.4

Quelle est la différence entre array_merge() et array_combine() ?

Réponse :

— `array_merge()` : Fusionne des tableaux

— `array_combine()` : Crée un tableau associatif à partir de deux tableaux

Exemple :

```php
// array_merge
$tab1 = ['a', 'b'];
$tab2 = ['c', 'd'];
$merge = array_merge($tab1, $tab2); // ['a', 'b', 'c', 'd']

// array_combine
$cles = ['nom', 'age'];
$valeurs = ['Ali', 25];
$combine = array_combine($cles, $valeurs);
// ['nom' => 'Ali', 'age' => 25]
```

4.4 Exercice 12 : Fonctions sur les fichiers

12.1

Comment ouvrir un fichier en PHP avec `fopen()` ?

Réponse :

Exemple :

```
// Ouverture en lecture
$fichier = fopen('document.txt', 'r');

// Ouverture en   criture
$fichier = fopen('document.txt', 'w');

// Ouverture en ajout
$fichier = fopen('document.txt', 'a');

// V rification
if ($fichier) {
    // Traitement
    fclose($fichier);
}
```

12.2

Expliquez l'utilisation des fonctions `fwrite()` et `fread()` pour manipuler des fichiers.

Réponse :

Exemple :

```
//   criture
$fichier = fopen('test.txt', 'w');
fwrite($fichier, "Hello World\n");
fwrite($fichier, "Deuxi me ligne\n");
fclose($fichier);

// Lecture
$fichier = fopen('test.txt', 'r');
$contenu = fread($fichier, filesize('test.txt'));
fclose($fichier);
echo $contenu;
```

12.3

Quels sont les risques de ne pas fermer un fichier ouvert après utilisation ?

Réponse : Les risques incluent :

- Fuites de mémoire
- Verrouillage du fichier
- Données non écrites (buffer)
- Limite du nombre de fichiers ouverts

12.4

Donnez un exemple où fgets() est utilisé pour lire un fichier ligne par ligne.

Réponse :

Exemple :

```
$fichier = fopen('fichier.txt', 'r');
if ($fichier) {
    while (($ligne = fgets($fichier)) !== false) {
        echo "Ligne : " . trim($ligne) . "\n";
    }
    fclose($fichier);
}
```